SDOMS: Experiment-based Project in Undergraduate Fundamentals of Programming

Nima Soltani
ni.soltani@mail.sbu.ac.ir

Alborz Seifaei
a.seifaei@mail.sbu.ac.ir

Dr.Mahdi Mojahedi
m_mojahedi@sbu.ac.ir

Abstract

This project is conceived as a pivotal component of the undergraduate curriculum to bridge the gap between theoretical computer science concepts and practical energy engineering challenges. It focuses on the design and simulation of a Smart Distribution Optimization & Management System (SDOMS). The project mandates the implementation of a robust object-oriented architecture to model Distributed Energy Resources (DERs). Additionally, it requires the application of recursive algorithms to verify basic system connectivity (DFS). The core operational stability is achieved through custom sorting algorithms for prioritized load shedding and bitwise logic for device state management. Finally, the integration of MATLAB for data visualization underscores the importance of cross-platform interoperability in modern engineering workflows.

Keywords— Smart Microgrid, Object-Oriented Programming, Recursive Algorithms, Load Shedding, MATLAB Visualization.

I. INTRODUCTION

In the contemporary landscape of power systems, the paradigm is shifting from centralized generation towards decentralized, smart microgrids. This transition is driven by the integration of renewable energy sources and the emergence of 'prosumers'. Efficient management of such systems is a computational challenge. The stability of a grid relies heavily on software agents capable of managing resources, verifying connections, and balancing supply and demand.

Consequently, algorithmic thinking is an essential skill for the modern energy engineer. This project, SDOMS, is designed to simulate the core logic of a microgrid controller. Students are tasked with building a system that models physical assets (solar panels, batteries, loads), verifies their connectivity using simple recursion, and prevents blackouts through intelligent load shedding mechanisms.

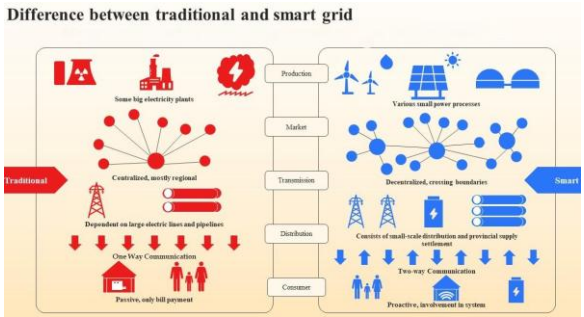

Fig. 1. Structural comparison between Traditional Centralized Grid (Left) and Smart Decentralized Grid (Right).

II. BACKGROUND

A.The Paradig mShift: From Vertical to Horizontal Integration

Traditional power systems ('Grid 1.0') operated on a rigid vertical hierarchy: GW-scale thermal plants generated power that was transmitted unidirectionally to passive consumers. This model is currently collapsing under the pressure of the 'Energy Trilemma': Security, Equity, and

Sustainability. The modern 'Smart Grid' integrates Information and Communication Technologies (ICT) with physical infrastructure. It replaces the unidirectional flow of electrons with a bidirectional flow of both power and data.

B.Microgrid Cyber-Physical Architecture

A Microgrid is defined by the IEEE 2030.7 standard as a group of interconnected loads and distributed energy resources within clearly defined electrical boundaries that acts as a single controllable entity with respect to the grid. It connects to the main utility grid at the Point of Common Coupling (PCC) but possesses the critical capability to island (disconnect) and operate autonomously during macrogrid faults.

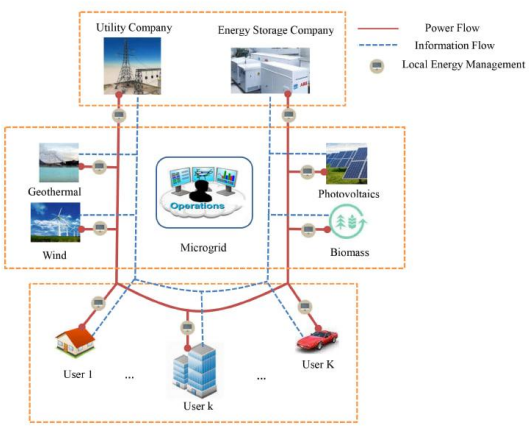

Fig. 2. Schematic diagram of a Microgrid integrating DERs, Storage, and Loads.

C. The Role of Energy Management Systems (EMS)

The EMS serves as the operational 'brain' of the microgrid. Unlike traditional SCADA systems which primarily monitor, an EMS performs active real-time optimization. It executes functions such as State Estimation, Economic Dispatch, and Resilience Analysis. In this project, the software agent developed by students acts as a Centralized EMS.

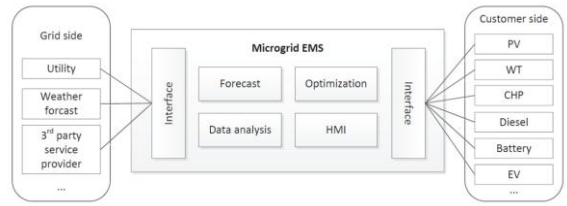

Fig. 3. Functional Architecture of a Microgrid EMS showing Inputs, Core Logic, and Control Interfaces.

D. Under-Frequency Load Shedding (UFLS) Logic

Grid stability dictates that Supply must equal Demand at every millisecond. In island mode, a microgrid loses the infinite inertia of the main grid. To prevent a total blackout when generation drops, the EMS must execute Under-Frequency Load Shedding (UFLS). This logic minimizes the 'Value of Lost Load' (VOLL) by prioritizing critical assets.

III. PROPOSED DESIGN AND IMPLEMENTATION DETAILS

The implementation is strictly divided into five phases. Adherence to the class structures, method signatures, and file formats is mandatory.

A. Phase I: Class Hierarchy & Data Ingestion

1. File Formats:

- devices.csv:

  Columns: ID, Type, Name, Value, Priority
  Type: 'S' for Source, 'L' for Load
  Priority: Integer (1=Critical, 2=High, 3=Normal). For Sources, this is always 0

- topology.csv:

  Columns: Source_ID, Target_ID
  Represents a directed edge from Source to Target

2. Class EnergyNode (Abstract Base Class):

- Attributes:

  node_id (int): Unique identifier
  name (str): Device name
  _status (int): A bit-field integer representing the device state

  Bit 0 (LSB): Connectivity (0: Disconnected, 1: Connected)
  Bit 1: Operation Mode (0: OFF, 1: ON)

- Methods:

  __init__(self, node_id, name): Constructor
  set_connected(): Uses bitwise OR (|) to set Bit 0 to 1
  turn_off(): Uses bitwise AND (&) with mask to set Bit 1 to 0
  is_on(): Returns True if Bit 1 is 1 (Check using bitwise AND)
  get_power(): Abstract method (raise NotImplementedError)
  __repr__(self): Returns string format [ID] Name: Status

3. Class Source (Inherits EnergyNode):

- Attributes: generation_capacity (float)
- Method get_power(): Returns +generation_capacity if device is ON, else returns 0

4. Class Load (Inherits EnergyNode):

- Attributes: demand_value (float), priority (int)
- Method get_power(): Returns -demand_value if device is ON, else returns 0

5. Data Parsing:

- Create a function load_devices(filename)
- Iterate through devices.csv. Instantiate Source or Load objects based on the 'Type' column
- Store objects in a global dictionary: network_objects = {id: object}
- Error Handling: Wrap the line parsing in a try-except block. If a line has missing values or invalid types, skip it and append the line number and raw text to error_log.txt

B. Phase II: Recursive Connectivity Verification

1. Graph Construction:

- Read topology.csv into an Adjacency Dictionary:
  graph = {node_id: [neighbor_ids...]}

2. Algorithm (DFS):

- Implement verify_grid_integrity(start_node_id, visited_set)
- Logic:

  1. Add start_node_id to visited_set
  2. Mark the corresponding object in network_objects as connected (call set_connected())
  3. Base Case: If start_node_id is the Main Grid (ID: 0), return True
  4. Recursive Step: For each neighbor in graph[start_node_id]:

     If neighbor not in visited_set:

     If verify_grid_integrity(neighbor, visited_set) is True, return True

  5. Return False if loop finishes without finding Node 0

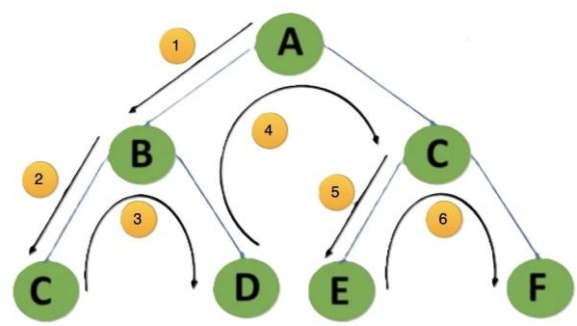

Fig. 4. Graph traversal using Depth First Search (DFS) to verify node connectivity.

C. Phase III: Controller Logic & Load Shedding

1. Sorting Algorithm (Bubble Sort):

- Implement a custom function sort_loads(load_list).

- Sorting Logic: The goal is to sort loads so that sheddable candidates come first.

  Primary Key: Priority (Descending). Priority 3 (Normal) should be at index 0. Priority 1 (Critical) at the end.
  Secondary Key (if Priorities are equal): Power Demand (Descending). Larger loads should be shedded first to resolve deficit quickly.

- Constraint: You must use nested loops (Bubble Sort or Selection Sort). Do not use Python's .sort().

2. Shedding Logic (The balance_grid function):

- Step 1: Calculate
  Net_Power = Sum(Source.get_power()) + Sum(Load.get_power()).
- Step 2: Check System Frequency/Balance.

  If Net_Power >= 0: System is stable.
  If Net_Power < 0: UFLS Triggered.

  Get all Active Loads (where is_on() is True).
  Sort them using sort_loads.
  Iterate through the sorted list:

  1. Call load.turn_off().
  2. Log the shedding event (Load ID, Priority).
  3. Recalculate Net_Power.
  4. Break loop if Net_Power >= 0.

D. Phase IV: Functional Data Analysis

After the control logic executes, perform analysis using functional programming paradigms.

1. Energy Not Served (ENS) Calculation:

- Use filter() with a lambda to extract a list of all Load objects where is_on() is False.
- Use map() to extract the demand_value from this filtered list.
- Use functools.reduce() (or simple sum) to calculate Total ENS.

2. Simulation Export:

- Write the final state to simulation_results.csv with columns:

  Timestamp (can be simulation step index).
  Total_Generation.
  Total_Demand (Requested).
  Actual_Consumption (After Shedding).
  Shedded_Amount_P1 (Priority 1).
  Shedded_Amount_P2 (Priority 2).
  Shedded_Amount_P3 (Priority 3).

E. Phase V: MATLAB Visualization Interface

1. Python Output:

- Ensure simulation_results.csv is created in the same directory.

2. MATLAB Script (visualize_grid.m):

- Data Import: Use
  T = readtable('simulation_results.csv');.
- Figure 1 (Power Balance):

  subplot(2,1,1): Plot Total_Generation and Total_Demand on the same axis using plot().
  Add a title, legend, and grid.

- Figure 2 (Shedding Analysis):

  subplot(2,1,2): Create a Bar Chart showing the total shedded power grouped by Priority.
  X-axis: Categorical {'Critical', 'High', 'Normal'}.
  Y-axis: Power (kW).
  This visualizes strictly how much load was sacrificed to save the grid.